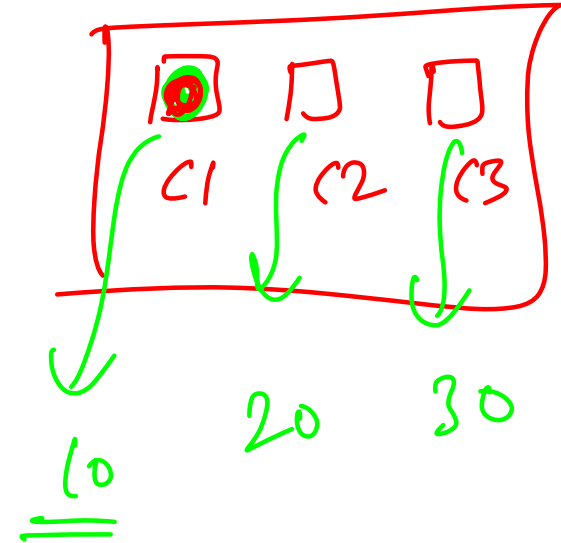


Dynamic Programming 1.1

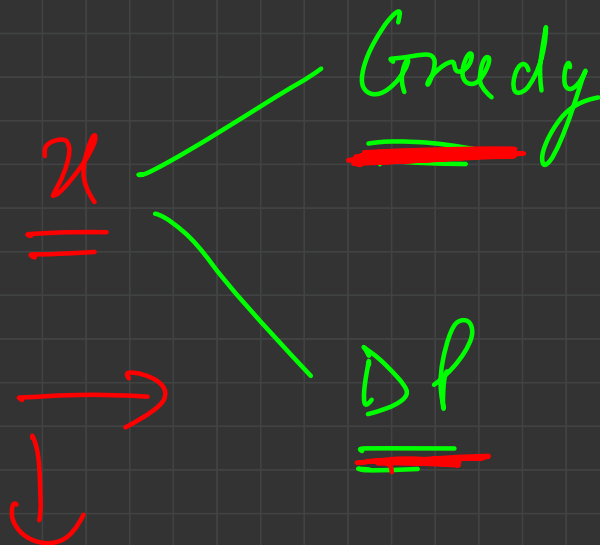
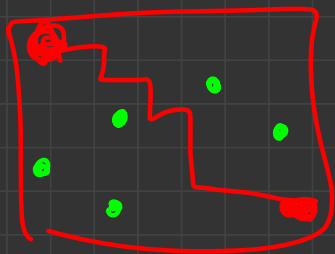
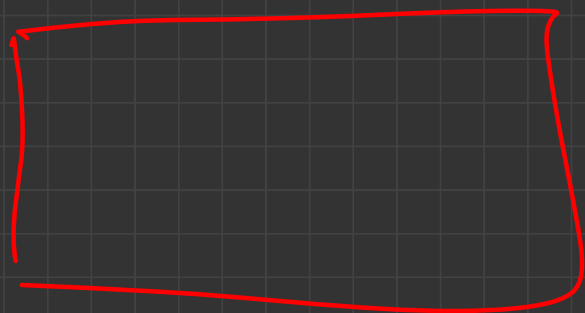
- Priyansh Agarwal

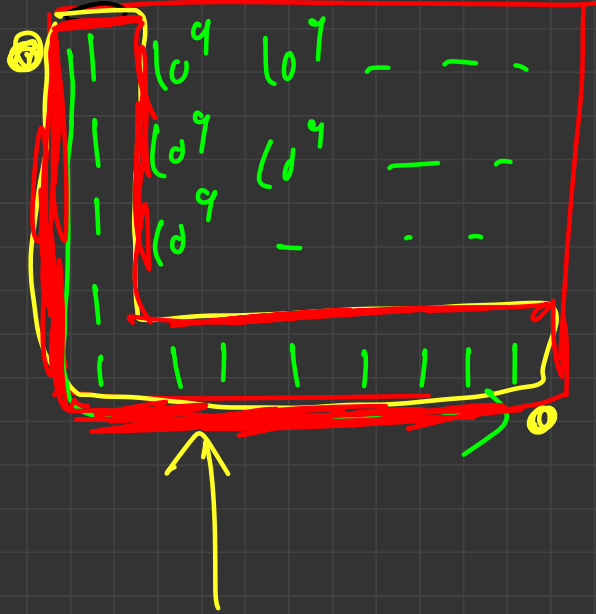
Why Dynamic Programming?

- Overlapping subproblems
- Maximize/Minimize some value
- Finding number of ways
- Covering all cases (DP vs Greedy)
- Check for possibility



$$\underline{\underline{25^2}} = \underline{\underline{625}}$$





Greedy



DP

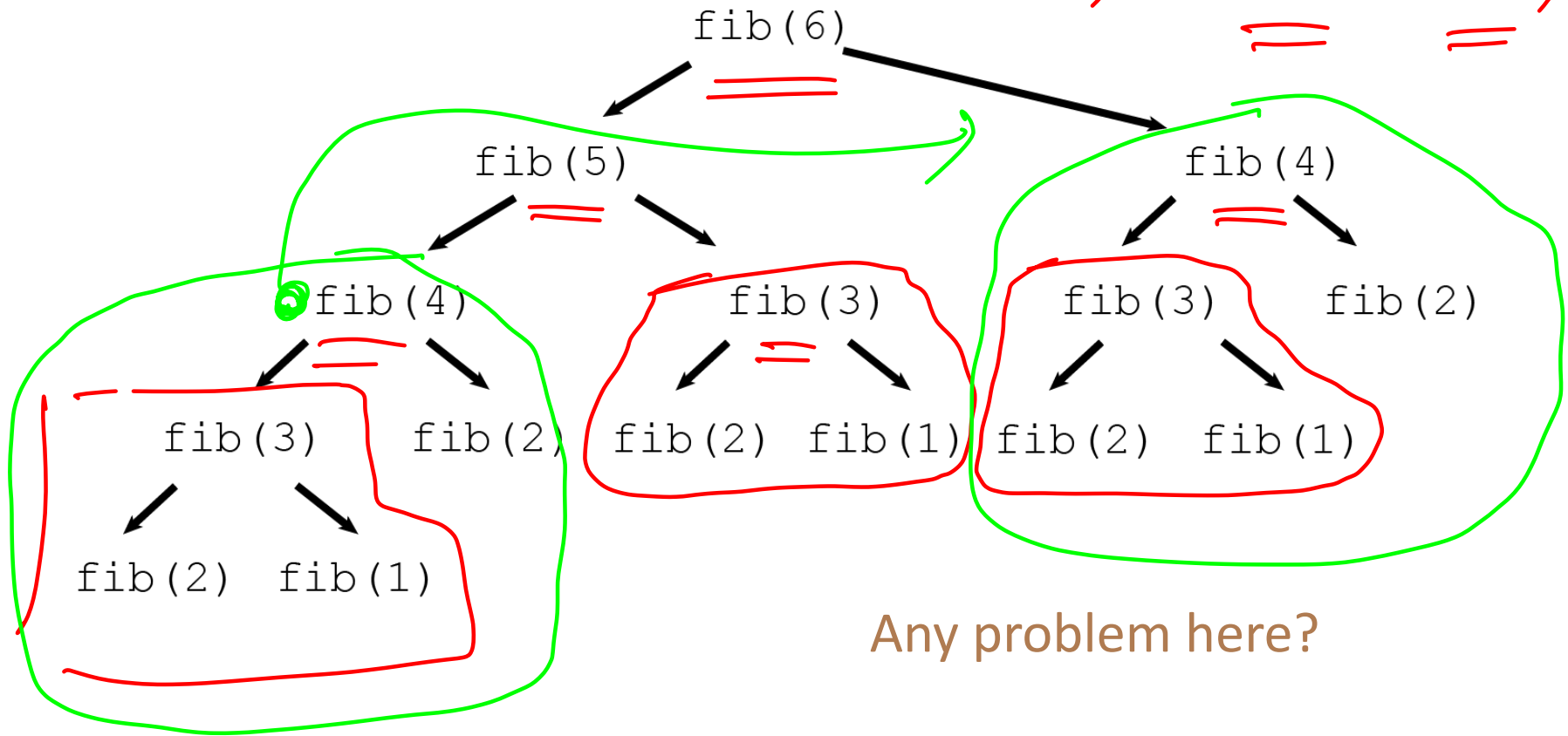
random
values filled
in grid

Need of DP

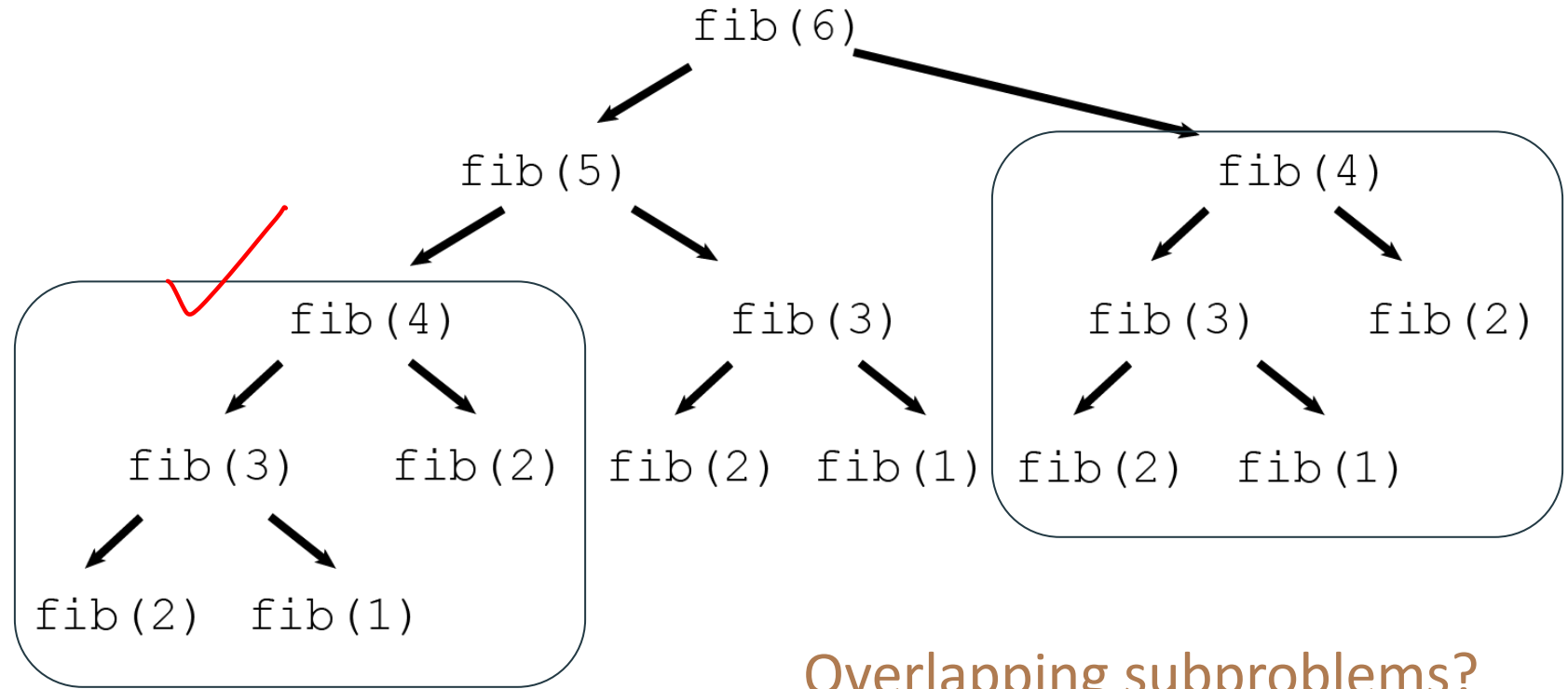
- Let's understand this from a problem
 - Find n^{th} fibonacci number
 - $F(n) = F(n - 1) + F(n - 2)$
 - $F(1) = F(2) = 1$

~~Handwritten scribbles in green and red ink.~~

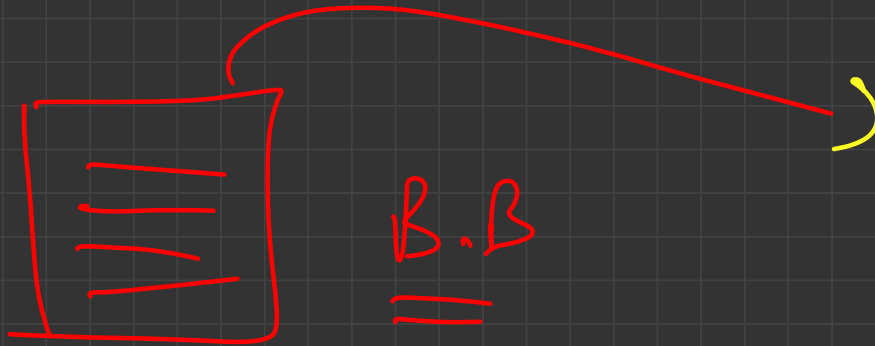
$$f(6) = f(5) + f(4)$$



Any problem here?

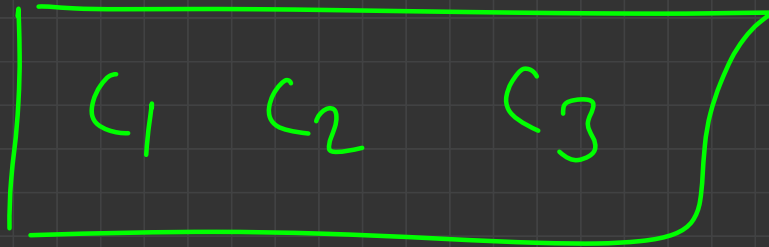


Memorization : Memorization



store it

some what



$\odot$	$\odot$	$\odot$
1	100	2
0	0	0
0	0	0

$1 \rightarrow x$
 $100 \rightarrow y$

S1 • x

S2 • y

S3 • z

S4 • a

S5 • b

$S1 \rightarrow x$

$S2 \rightarrow y$

$S3 \rightarrow z$

F(6)

- $\frac{1}{2} \times \frac{1}{2} = \frac{1}{4}$

find answer →

Sus
problem

exists?

No

Yes

Stop ← find
Ans

return

→ Return

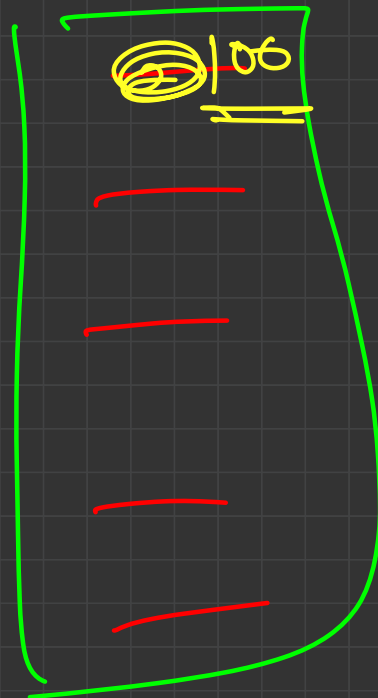
S1 ←

S2

S3

S4

S5



Calculated?

Ans



$f(6)$ $f(5)$ $f(4)$ $f(3)$ $f(2)$ $f(1)$

6

5

4

3

2

1

Calculated

0

0

0

0

1

0

1

0

1

6

5

4

3

2

1

Ans

—

—

—

2

1

1

Ans

$(-100, 100)$

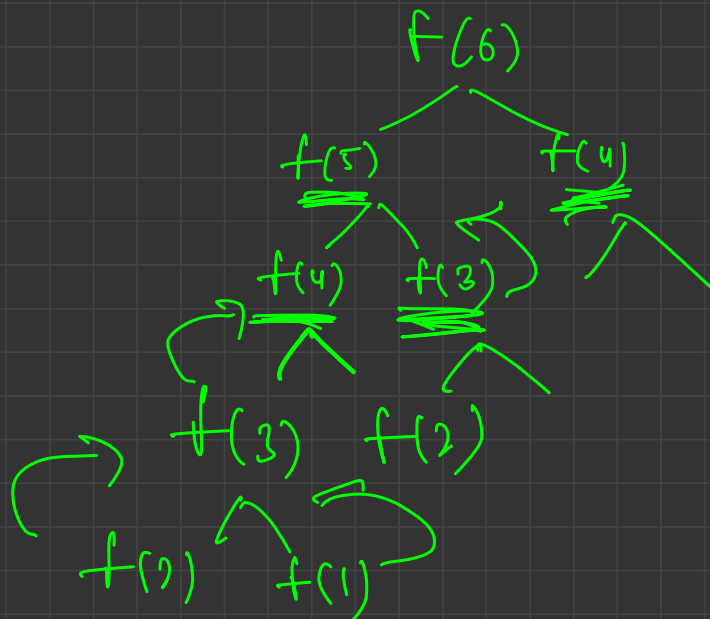
$$\begin{bmatrix} -1 & -1 & -1 & -1 & -1 & -1 \end{bmatrix}$$

1 2 3 4 5 6

Default Value

Aus —,

1	1	1	1	2	3	5	8
1	2	3	4	5	6		



Without DP

```
int functionEntered = 0;
int helper(int n){
    functionEntered++;
    if(n == 1 || n == 2){
        return 1;
    }
    return helper(n - 1) + helper(n - 2);
}
void solve(){
    int n;
    cin >> n;
    cout << helper(n) << nline;
    cout << functionEntered << nline;
}
```

Handwritten notes:

- $f(n)$ (next to the `helper` function)
- $f(n-1)$ and $f(n-2)$ (next to the recursive calls)
- $O(n)$ (written on the left, with a bracket indicating the time complexity of the `helper` function)

functionEntered = 1664079
with n = 30

Handwritten notes:

- Red underlines under the value 1664079 and the input 30.
- A large red checkmark or 'X' mark below the text.

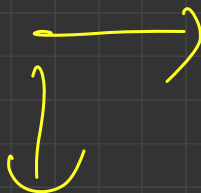
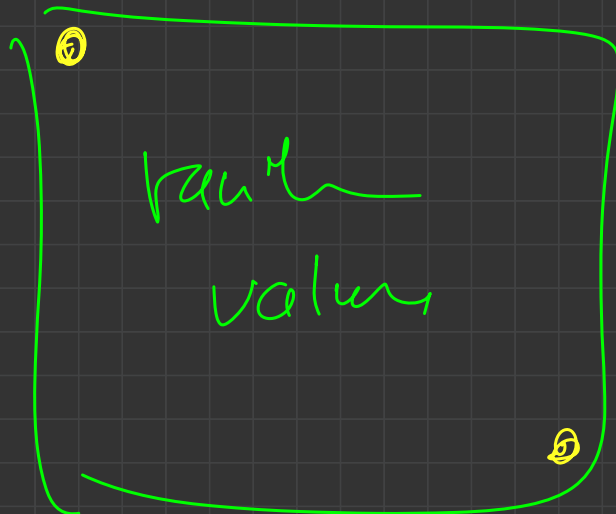
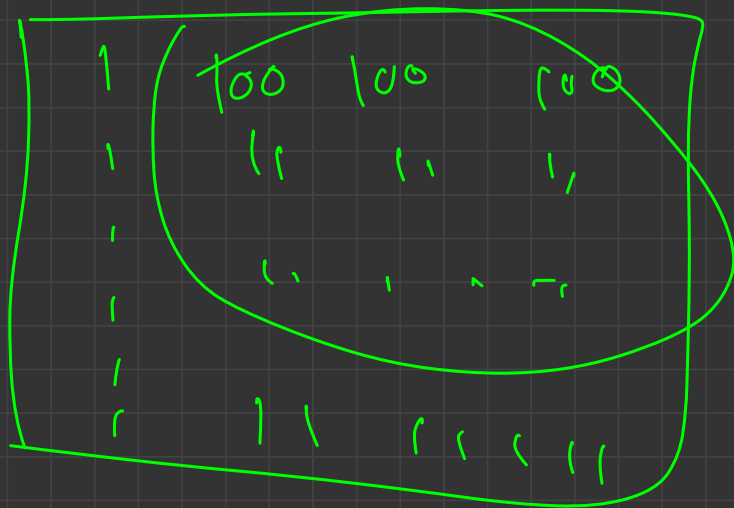
With DP

```
int functionEntered = 0;
int dp[40];
int helper(int n){
    functionEntered++;
    if(n == 1 || n == 2){
        return 1;
    }
    if(dp[n] != -1)
        return dp[n];
    return dp[n] = helper(n - 1) + helper(n - 2);
}
void solve(){
    int n;
    cin >> n;
    for(int i = 0; i <= n; i++)
        dp[i] = -1;
    cout << helper(n) << nline;
    cout << functionEntered << nline;
}
```



The diagram shows a green box containing the sequence of values in the dp array: [-1, -1, -1, -1, -1]. This represents the state of the dp array for indices 0 through 4. Green arrows indicate the recursive calls: from n=5 to n=4 and n=3, and from n=4 to n=3 and n=2.

functionEntered = 57
with n = 30



Let's solve another problem!

Given a 2D grid (N X M) with numbers written in each cell, find the path from top left (0, 0) to bottom right (n - 1, m - 1) with minimum sum of values on the path

$$\begin{aligned} &(-100 \times (m+n)) \\ &+ 100 \end{aligned}$$

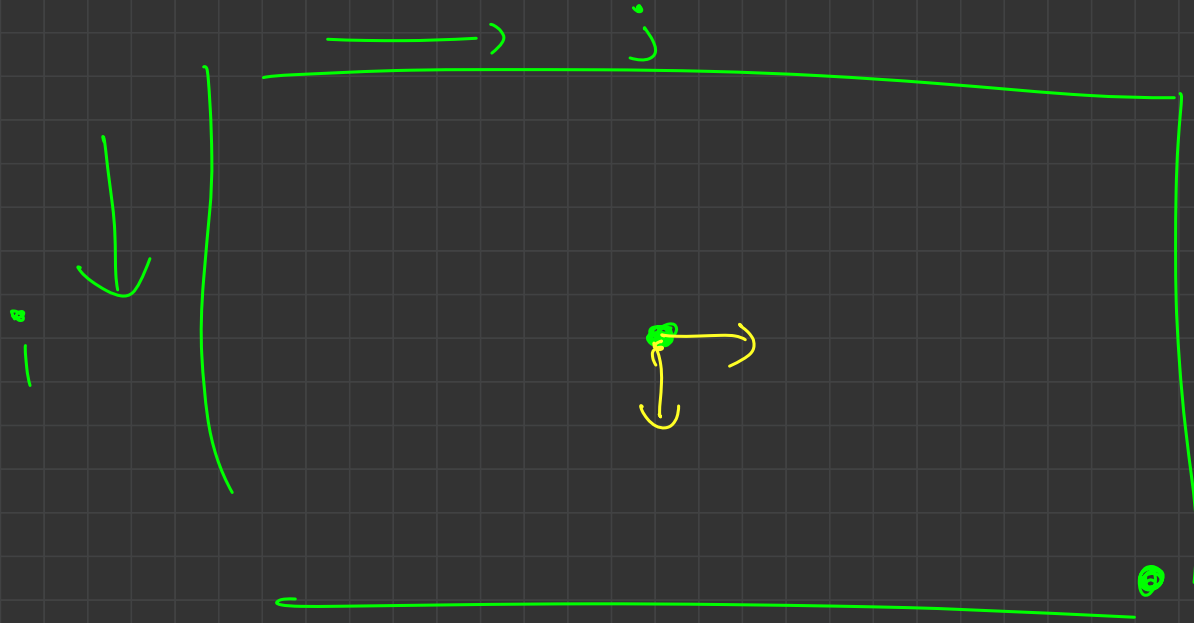
1	5	8
6	2	7
9	3	4

$$\underline{\underline{(-100, 100)}}$$



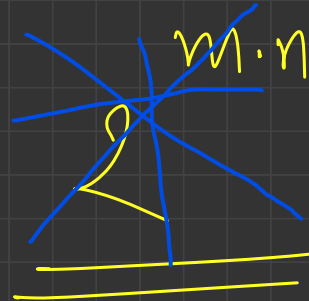
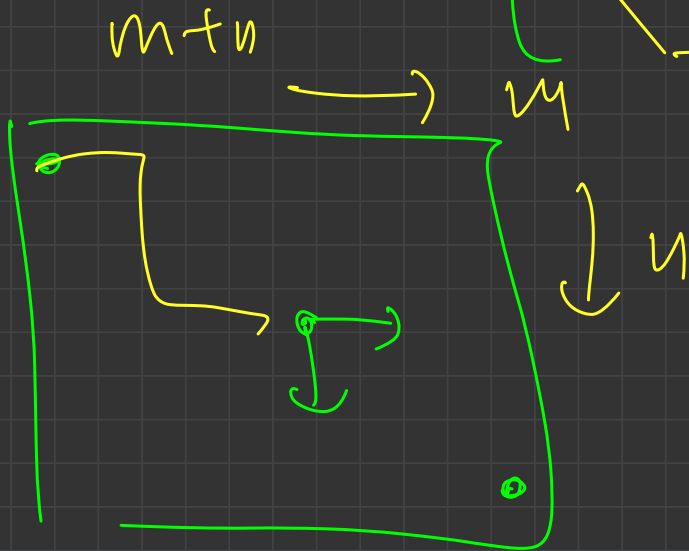
$$(n) \rightarrow 20$$

$$(m) \rightarrow 20$$

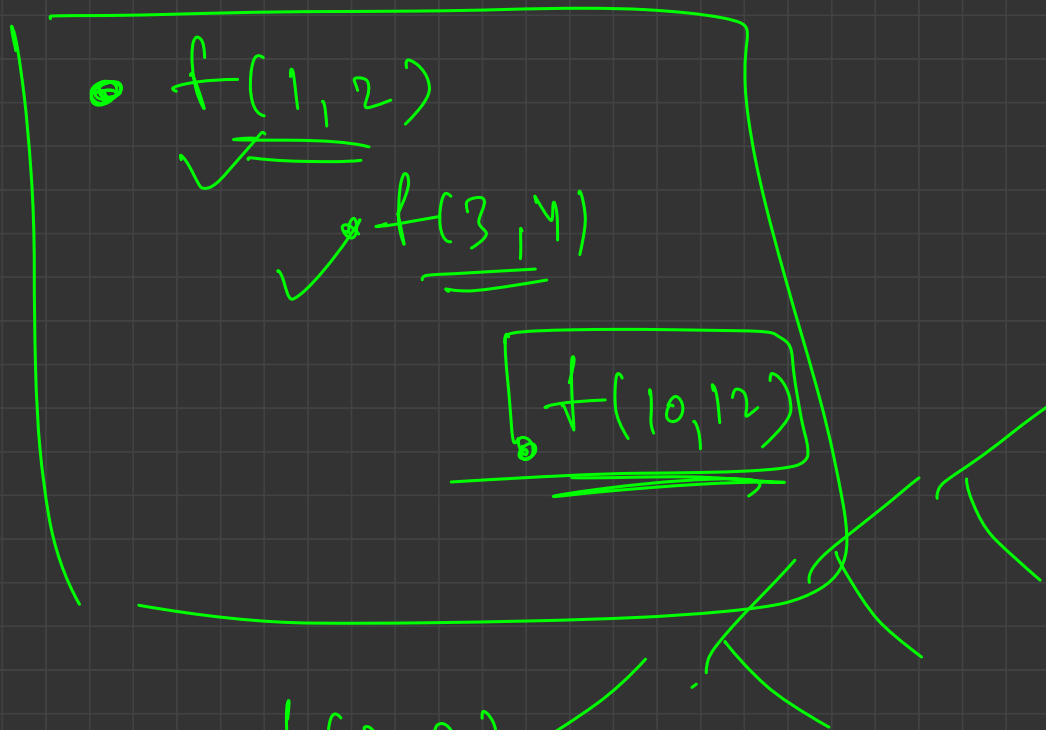


$f(i, j)$ = minimum sum path from (i, j)
to bottom right

$$f(i, j) = \min \begin{cases} f(i+1, j) + \text{grid}[i][j] \\ f(i, j+1) + \text{grid}[i][j] \end{cases}$$

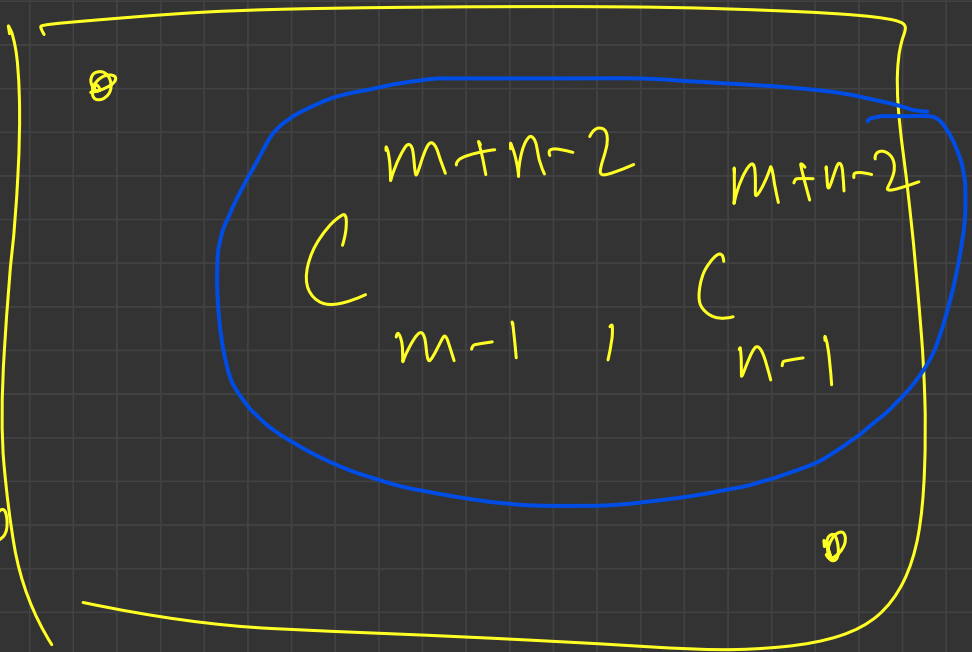


$$\underline{\underline{2^{m+n}}}$$



$$\underline{f(1,2)} < \begin{matrix} f(2,2) \\ f(1,3) \end{matrix}$$

$k-1$ steps



$m-1$ steps

$\underline{\underline{D R D D R R R D D D}}$

$R \rightarrow$

$\underline{\underline{m-1}}$

$\downarrow$
 $\underline{\underline{n-1}}$

$a a a a a a \quad 6$

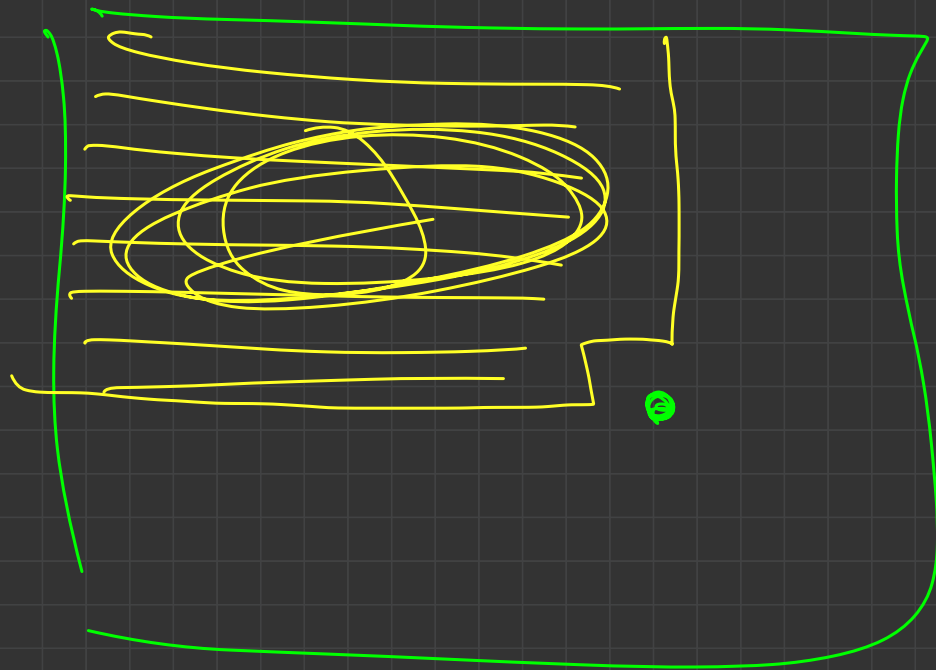
$b b b \quad 3$

$(6+3)!$

$\hline 6! 3!$

$6 \frac{(m+n-2)!}{(n-1)!(m-1)!}$

$\stackrel{m+n-2}{=} C_{n-1}$



$$\underline{\underline{f(100, 200)}}$$

$$\underline{\underline{100 \times 200}}$$

$$\underline{\underline{20,000}}$$

Naive Way

Explore all paths. Standing at (i, j) try both possibilities $(i + 1, j)$, $(i, j + 1)$

Every cell has two choices

Time complexity: $O(2^{m \cdot n})$?

Actual Time complexity: $O(C(n + m - 2, m - 1))$

Handwritten notes and diagrams illustrating the complexity analysis:

- A box containing 2^{m+n} with an arrow pointing to 2^{m+n-2} .
- A crossed-out line below the box.
- A blue circle containing C^{m+n-2}_{m-1} with an arrow pointing from the crossed-out line.

• (2, 3)

• (5, 6)

$$f(6) \rightarrow 6$$

(i, j)

dp(n, m)

Efficient Way

Overlapping subproblems

Memoization

Time complexity: $O(n * m)$

Space complexity: $O(n * m)$

```
int grid[n][m]; // input matrix
```

```
int dp[n][m]; // every value here is -1
```

```
// subproblem: f(i, j) represents minimum sum path from (i, j) to (n - 1, m - 1)
```

```
int f(int i, int j){ //
```

```
    if(i >= n || j >= m){ // moving outside the grid // not allowed  
        return INF;
```

```
    }
```

```
    if(i == n - 1 && j == m - 1) // reached the destination  
        return grid[n - 1][m - 1];
```

```
    if(dp[i][j] != -1) // this state has been calculated before  
        return dp[i][j];
```

```
    // state never calculated before
```

```
    dp[i][j] = grid[i][j] + min(f(i, j + 1), f(i + 1, j));  
    return dp[i][j];
```

```
}  
  
void solve(){
```

```
    cout << f(0, 0) << endl;
```

```
}
```

0 based indexing

f(i, j)



```
int grid[n][m]; // input matrix
```

```
int dp[n][m]; // every value here is -1 (countes = 0)
```

```
// subproblem: f(i, j) represents minimum sum path from (i, j) to (n - 1, m - 1)
```

```
int f(int i, int j){ // (countes + 1)
```

```
    if(i >= n || j >= m){ // moving outside the grid // not allowed  
        return INF;  
    }
```

```
    if(i == n - 1 && j == m - 1) // reached the destination
```

```
        return grid[n - 1][m - 1];
```

```
    if(dp[i][j] != -1) // this state has been calculated before
```

```
        return dp[i][j];
```

```
    // state never calculated before
```

```
    dp[i][j] = grid[i][j] + (min(f(i, j + 1), f(i + 1, j)));
```

```
    return dp[i][j];
```

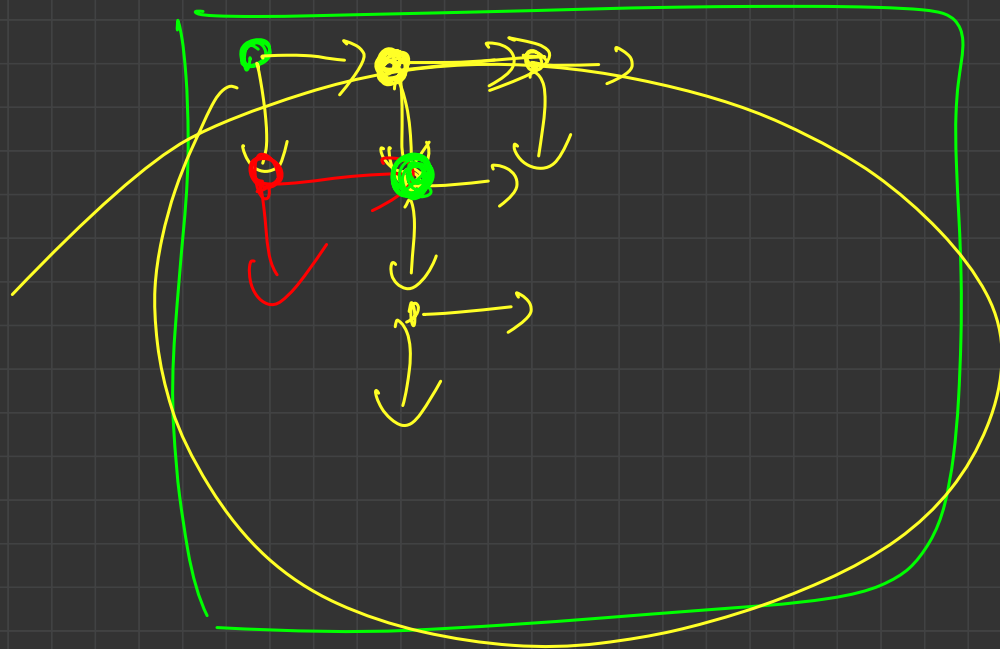
```
}
```

```
void solve(){
```

```
    cout << f(0, 0) << endl;
```

```
}
```

K (counter incr) work done in 1 time



m.n

Important Terminology

State: A subproblem that we want to solve. The subproblem may be complex or easy to solve but the final aim is to solve the final problem which may be defined by a relation between the smaller subproblems. Represented with some parameters.

*n*th fibonacci $f(n)$
=

Transition: Calculating the answer for a state (subproblem) by using the answers of other smaller states (subproblems). Represented as a relation b/w states.

$$f(n) = \underline{\underline{f(n-1) + f(n-2)}}$$

Transition Equation

$$\underline{\underline{L.H.S}} = \underline{\underline{R.H.S}}$$

$$\underline{\underline{f(n)}} = \underline{\underline{f(n-1)}} + 2 \underline{\underline{f(n-2)}} + 5 \underline{\underline{f(n-5)}}$$

Exercise

$$f(i, j) = \underline{\underline{dp(i)(j)}}$$

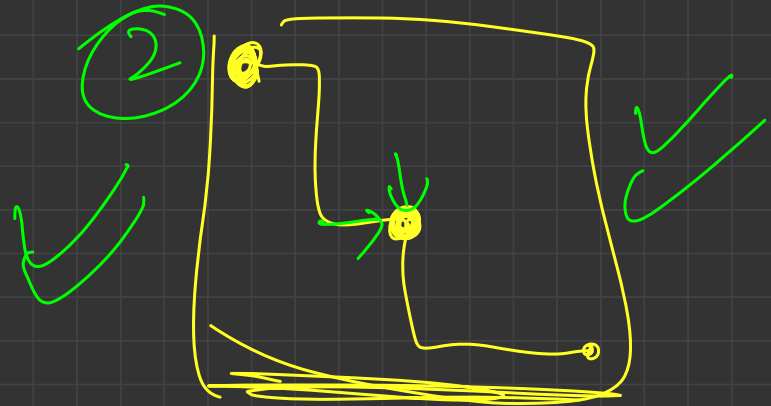
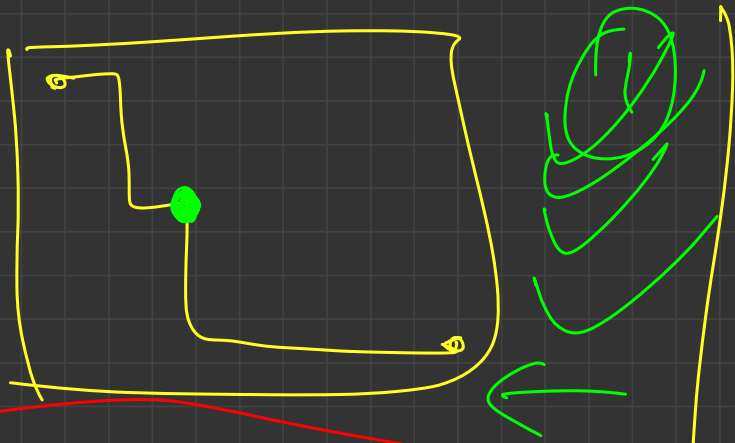
Fibonacci Problem:

- State

- dp[i] or f(i) meaning ith fibonacci number

- Transition

- dp[i] = dp[i - 1] + dp[i - 2]



$dp(i,j)$ is the min
sum path from (i,j)
to $(n-1, m-1)$

$dp(i,j)$ is the
min sum path
from $(0,0)$ to (i,j)

$$dp(i,j) = grid[i][j] + \min \{ dp(i,j+1), dp(i+1,j) \}$$

dp(0,0)

$$dp(i,j) = \text{grid}(i,j) + \min \begin{cases} dp(i-1,j) \\ dp(i,j-1) \end{cases}$$

top down }
bottom up }

dp(n-1,m-1)

State : dp(i,j) ✓

f(i,j) ✓

dp(row,col) ✓

ij ✓

dp(i,j) = _____ ✗

✓ dp(i,j) = min cost of path from
f(i,j) (i,j) to bottom right

Exercise

Matrix Problem:

- State
 - $dp[i][j]$ = shortest sum path from (i, j) to $(n - 1, m - 1)$
- Transition
 - $dp[i][j] = grid[i][j] + \min(dp[i + 1][j], dp[i][j + 1])$

Final suspoalun
 $dp[0][0]$ ↙

Let's solve another problem

Given an array of integers (both positive and negative). Pick a subsequence of elements from it such that no 2 adjacent elements are picked and the sum of picked elements is maximized.

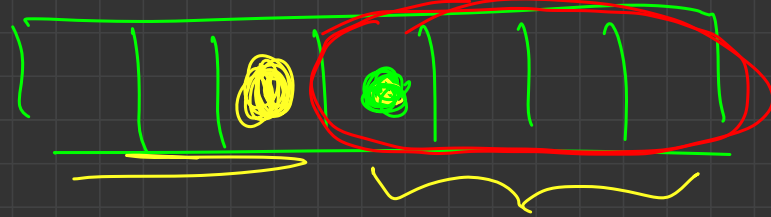
1	4	2	-10	10	5
---	---	---	-----	----	---

Sum = 14

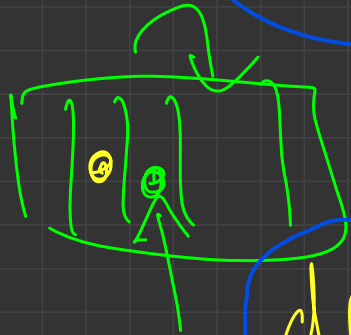
1	4	2	-10	10	5
---	---	---	-----	----	---

Sum = 13

State :



$dp[i][0]$ = max sum from $(i \text{ to } n-1)$
Such that $(i-1)$ was not picked



$dp[i][1]$ = max sum from $(i \text{ to } n-1)$
Such that $(i-1)$ was picked

→ $dp[i+1][0]$

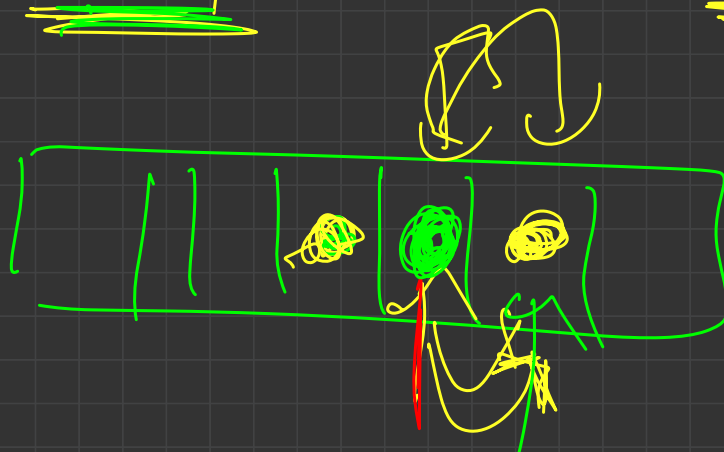
dp[i][j]

pick up

not pick up

dp[i+1][j] + a[i]

dp[i+1][j]



$$\checkmark \quad dp[i][0] = \max(dp[i+1][1] + a[i], dp[i+1][0])$$

$$\checkmark \quad dp[i][1] \Rightarrow dp[i+1][0]$$

$$\text{final} \rightarrow dp[0][0], \quad \text{max}(dp[0][0], \quad \text{dp[0][1]})$$

Some ways to solve the problem

1. Having 2 parameters to represent the state

State:

$dp[i][0]$ = maximum sum in (0 to i) if we don't pick i^{th} element

$dp[i][1]$ = maximum sum in (0 to i) if we pick i^{th} element

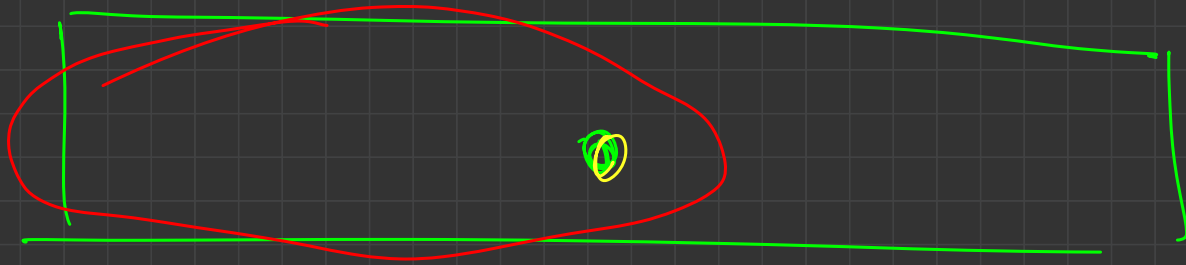
Transition:

$dp[i][0] = \max(dp[i - 1][1], dp[i - 1][0])$

$dp[i][1] = arr[i] + dp[i - 1][0]$

Final Answer:

$\max(dp[n - 1][0], dp[n - 1][1])$



$$\underline{\underline{dp(n-1)(0)}}$$

$$\underline{\underline{dp(i)(0)}} = \max \begin{cases} dp(i-1)(0) \\ dp(i-1)(1) \end{cases}$$

$$dp(n-1)(1)$$

$$dp(i)(1) = dp(i-1)(0) + a[i]$$

Some ways to solve the problem

2. Having only 1 parameter to represent the state

State:

$dp[i]$ = max sum in (0 to i) not caring if we picked i^{th} element or not

Transition: 2 cases

- pick i^{th} element: cannot pick the last element : $arr[i] + dp[i - 2]$
- leave i^{th} element: can pick the last element : $dp[i - 1]$

$$dp[i] = \max(arr[i] + dp[i - 2], dp[i - 1])$$

Final Answer:

$dp[n - 1]$

```
int a[n]; // input array

int dp[n]; // filled with -INF to represent uncalculated state
// f(i) = max sum till index i
int f(int index){
    if(index < 0) // reached outside the array
        return 0;
    if(dp[index] != -INF) // state already calculated
        return dp[index];

    // try both cases and store the answer
    dp[index] = max(a[index] + f(index - 2), f(index - 1));
    return dp[index];
}

void solve(){
    cout << f(n - 1) << endl;
}
```

Handwritten notes:
- Red and green underlines above the `if` statements in the `f` function.
- $INF = 1e18$ written in green.

Time and Space Complexity in DP

Time Complexity:

Estimate: Number of States * Transition time for each state

Exact: Total transition time for all states

Space Complexity:

Number of States * Space required for each state

$dp(i) \rightarrow \left\{ \begin{array}{l} n/2 \end{array} \right\}$ steps

$dp(10) = n/10$

$dp(1) = 1$

$dp(n) = n/n = 1$

transition
steps

$$df(1) = \underline{\underline{n}}$$

$$df(2) = n/2$$

$$df(3) = n/3$$

⋮

$$df(n) = n/n \text{ steps}$$

$$n + \frac{n}{2} + \frac{n}{3} + \dots + \frac{n}{n}$$

$$\underline{\underline{n \left(1 + \frac{1}{2} + \frac{1}{3} + \dots + \frac{1}{n} \right)}}$$

$$\left| \frac{1}{1} + \frac{1}{2} + \frac{1}{3} + \dots + \frac{1}{n} \right| < \log n$$

$$\underline{\underline{n \log n}}$$

$$dp(i) = dp(i-1) + dp(i-2)$$

states $\leftarrow$ $O(n)$ } $\rightarrow$ $O(n)$

T.T $\approx O(1)$

$$dp(i, j) = grid(i, j) + \min \begin{cases} dp(i-1, j) \\ dp(i, j-1) \end{cases}$$

$$O(n, m) \cdot O(1) \approx \underline{O(n, m)}$$

$$df(i, j) = \longrightarrow$$

$$O(1)$$

$$dp[i][j] = \underline{\underline{O(1)}}$$

$$O(n \cdot 2) \cdot O(1) = \underline{\underline{O(n)}}$$

